



GarageST Developer Handbook

Engineering Architecture & Implementation

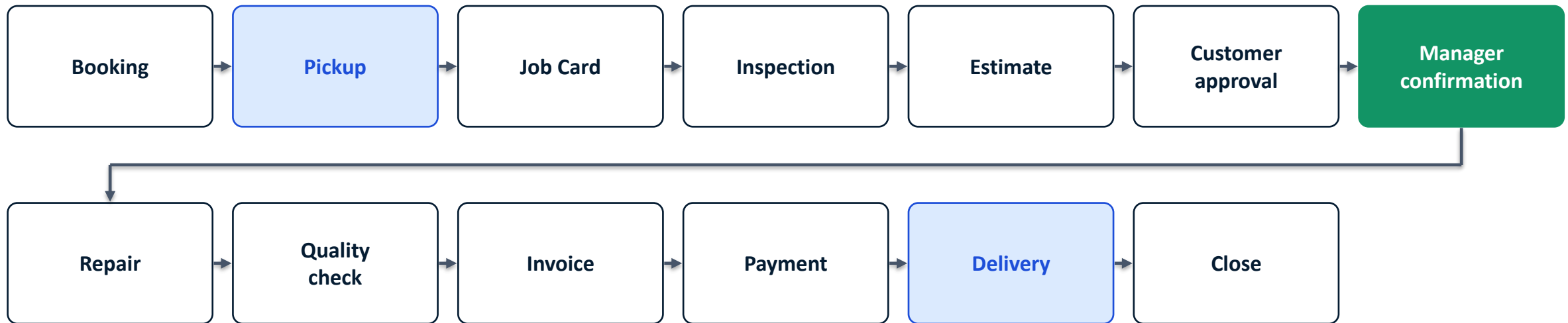
Flutter + Spring Boot + PostgreSQL + Cloudflare R2 + Firebase FCM

Living engineering reference · 2026-10-02 · TVASTRI

Source of truth: GARAGEST_DEVELOPER_HANDBOOK.md · Contains variable names only, no secrets



End-to-end vehicle service lifecycle



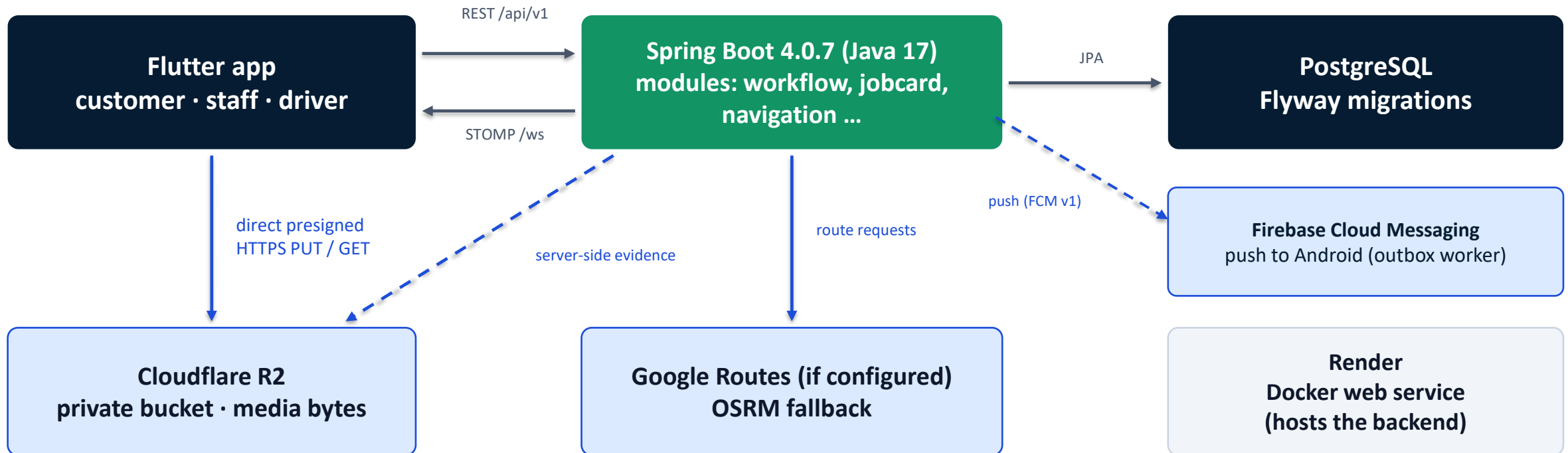
WHO USES IT



- Booking, pickup and delivery run as trips: live map, OTP-verified handover and photo evidence.
- The job card moves through a strict state machine owned by the backend; Flutter only presents that state.
- Blue = trip-driven steps (pickup, delivery). Green = the explicit manager gate.



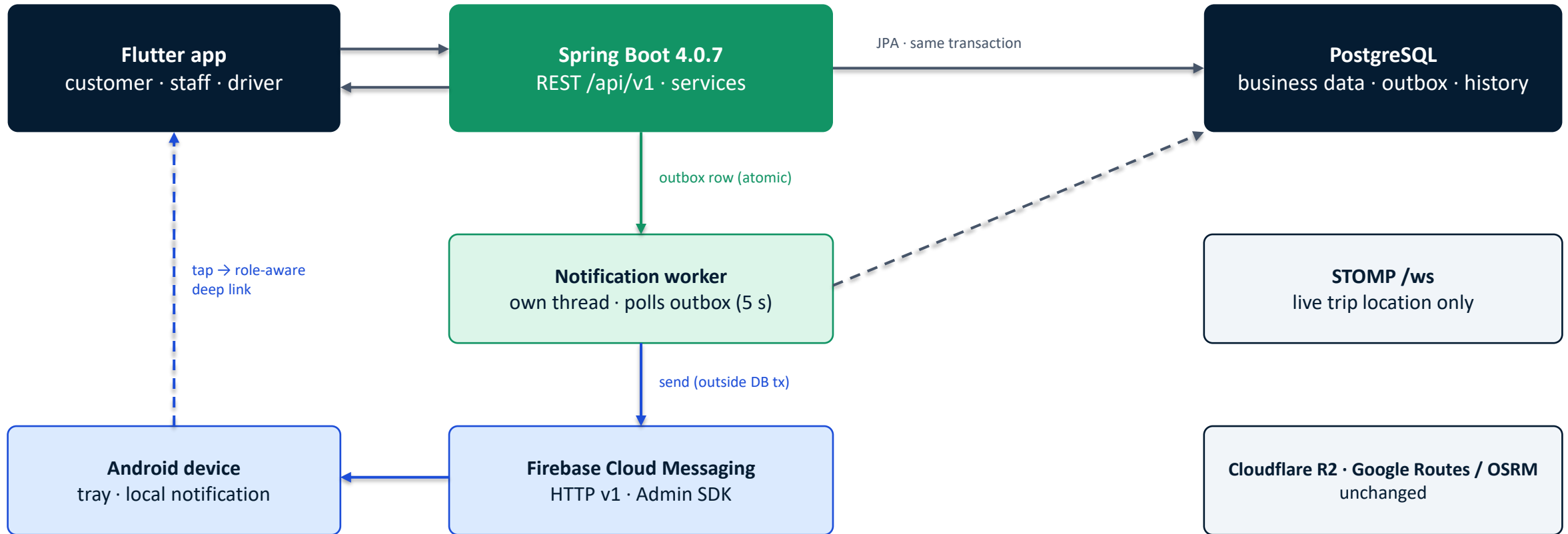
System at a glance



The backend owns business state, validation and authorization. Flutter renders that state and never invents it. Media bytes never pass through Spring (direct-to-R2).



GarageST architecture with notifications



WebSocket/STOMP is not used for notifications. No Kafka: a PostgreSQL transactional outbox gives durability. FCM is delivery only; PostgreSQL is the source of truth.



Technology stack

Java 17
backend runtime

Spring Boot 4.0.7
web · security · JPA ·
WebSocket

PostgreSQL
system of record

Flyway
migrations V1 ... V62

Flutter / Dart
provider · flutter_map ·
google_maps

STOMP / WebSocket
live driver location

Cloudflare R2
presigned direct upload

Google Routes / OSRM
routing + fallback

Media3 Transformer 1.4.1
on-device video evidence

Docker + Render
eclipse-temurin:17 · backend
hosting

Firebase Cloud Messaging
Admin SDK 9.9.0 ·
firebase_messaging

No Kafka
not used anywhere



Repository and module structure

Flutter · D:\garagest_flutter

- lib/app — composition root (app.dart), routes
- lib/core — ApiClient, EnvConfig, secure storage, location
- lib/features/<feature>/{models, services, screens, state, widgets}
- workflow · jobcard · estimate · repair · booking · navigation · handover · media · customer_portal · auth
- android/.../kotlin — MainActivity, EvidenceVideoProcessor
- test/ mirrors lib/

Backend · D:\garageos-backend

- core — enums, exceptions, security, audit base
- modules/<module>/{controller, service(+impl), repository, entity, dto, mapper, validator}
- serviceworkflow · jobcard · estimate · repairtask · jobassignment · invoice · delivery
- booking · navigation · handover · media · customer · identity · audit
- db/migration — Flyway V{n}__description.sql
- Dockerfile — Render build

Feature-first layout: services take ApiClient by constructor and are registered once in app.dart; screens never construct services.



Responsibility boundaries

Backend	Flutter	PostgreSQL	Cloudflare R2
Business truth	Presentation + projection	Source of truth	Media bytes
state machines, validation, authorization, garage scoping	renders backend state; derives the workflow step from completedSteps; role gating is cosmetic	persisted state, Flyway-managed schema, ddl-auto=validate	private bucket; presigned URLs only; trip media is NOT here

Rule: Flutter never advances a workflow step locally past a mutation that refreshes backend status. After the backend answers, the step is re-derived from completedSteps.



Security architecture

JWT bearer
access ~1 h, refresh ~7 d; auth endpoints
public

Role authorization
@PreAuthorize on controllers (MANAGER,
SERVICE_ADVISOR, OWNER sets)

Garage scoping
authorizeJobCardGarage,
authorizeDeliveryAction, trip staff checks

Manager gate
JobCardServiceImpl.authorizeProceedToRepair
— MANAGER only

Technician scope
only own non-cancelled assignment in own
garage

Driver ownership
?driverId must equal the caller; active DRIVER
of the garage

Customer ownership
not-found instead of forbidden

OTP
hashed code, 10 min expiry, 5 attempts

Private R2
short-lived presigned URLs; server-generated
keys

Flutter UI gating (Permissions, step roles, allowedActions) hides buttons only. It is NOT a security boundary — the backend re-checks every rule that matters.

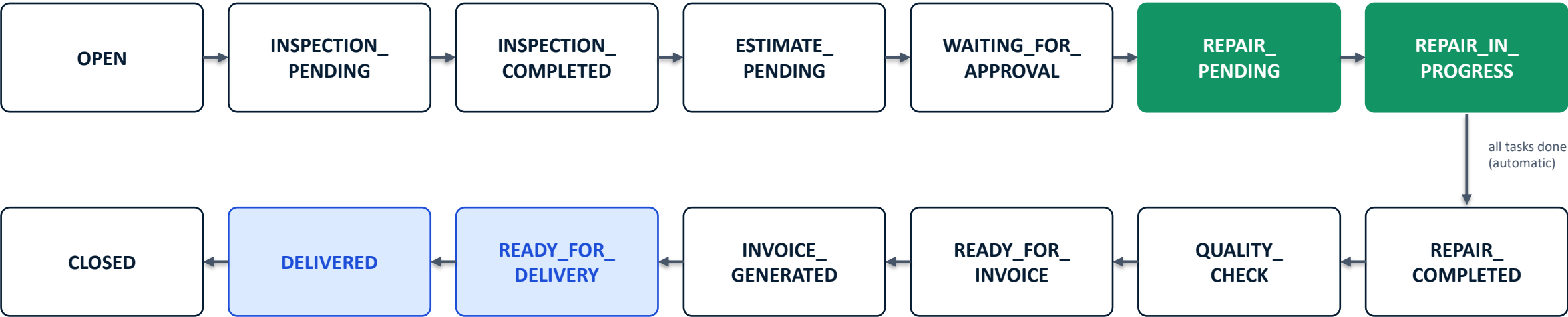


Who owns the state?

State	Owner (writer of record)	Authorized by
JobCard.status	JobCardServiceImpl (+ Estimate, RepairTask/JobAssignment, QC, Invoice, Delivery services) — all via JobCardStatusValidator	per operation
Estimate.status	EstimateServiceImpl, EstimateItemServiceImpl	customer owner (approve/reject)
RepairTask / JobAssignment	RepairTaskServiceImpl, JobAssignmentServiceImpl	assigned technician / assignment roles
Booking.status	BookingServiceImpl; JobCardServiceImpl sets COMPLETED	customer / staff
NavigationRequest.status	NavigationRequestServiceImpl (REQUESTED); NavigationTripServiceImpl (ASSIGNED, IN_PROGRESS, COMPLETED)	garage staff / driver
NavigationTrip	NavigationTripServiceImpl	the driver (requireCallerIsDriver)
VehicleHandover	HandoverServiceImpl	customer generates, driver verifies
Delivery.status	DeliveryServiceImpl (only DELIVERED is written)	staff or verified delivery trip
Workflow completedSteps	ServiceWorkflowServiceImpl.getWorkflowStatus — pure projection of JobCardStatus	read-only
Flutter currentStep	WorkflowController — derived from completedSteps	none (presentation)



JobCard state machine



Manager gate: REPAIR_PENDING → REPAIR_IN_PROGRESS is written ONLY by JobCardServiceImpl.startRepair

LEGACY (do not write)

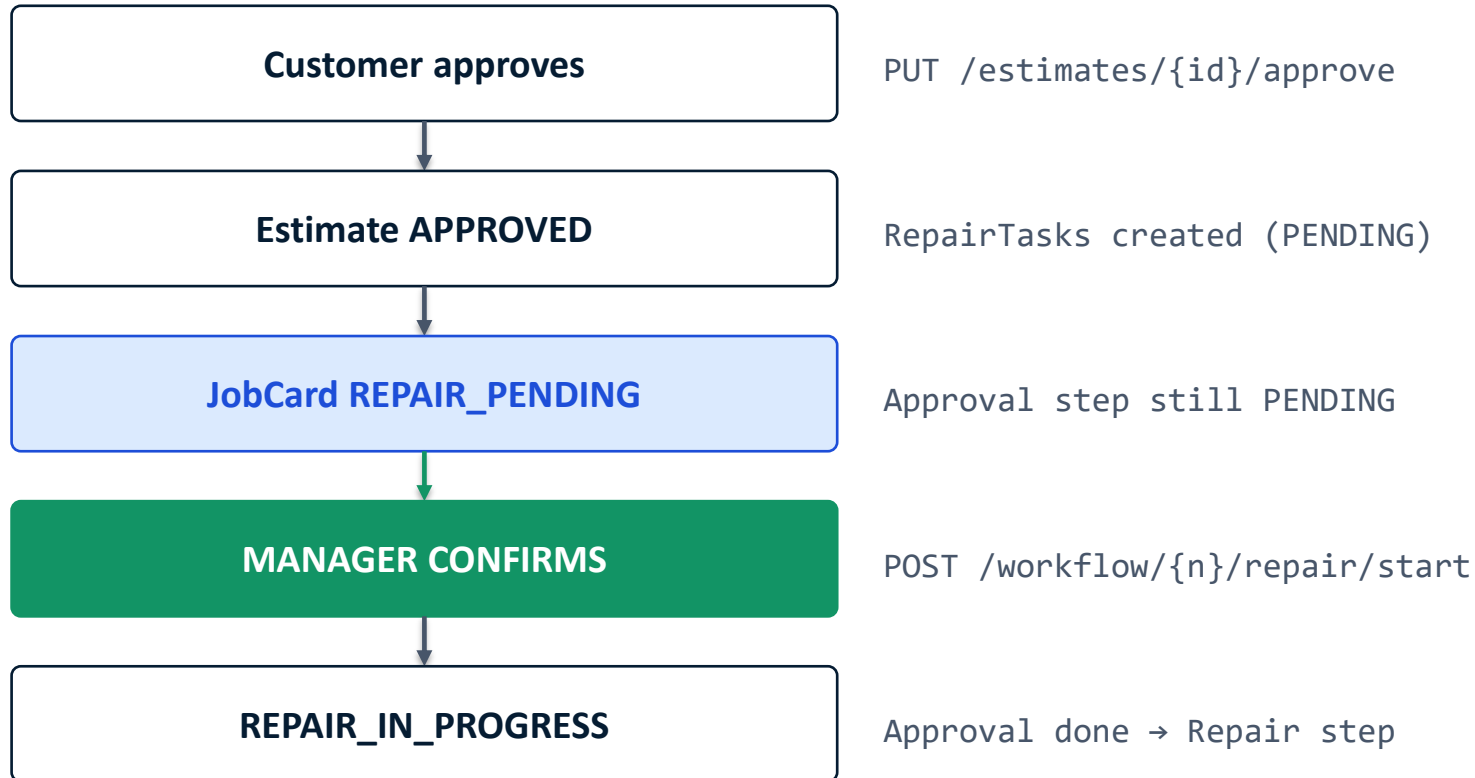


CANCELLED is reachable from most pre-delivery states. Rework: REPAIR_COMPLETED / QUALITY_CHECK → REPAIR_PENDING (manager confirms again).

Legacy source states are kept so old rows can still move forward through the validator (e.g. ESTIMATE_APPROVED → REPAIR_PENDING).



Customer approval ≠ manager confirmation



BLOCKED

- Technician cannot start a task while the job is REPAIR_PENDING
- JobAssignmentServiceImpl.startJob and RepairTaskServiceImpl.startRepair both refuse
- start_repair action is hidden from non-managers
- MANAGER role + estimate APPROVED required; idempotent

Confirmation is an explicit manager action. The pending state is REPAIR_PENDING as interpreted by the workflow projection (no new JobCard status).



The workflow bug and the fix

BEFORE

- Customer approval → job REPAIR_PENDING
- resolveCompleted() added APPROVAL at REPAIR_PENDING
- WorkflowController jumped to the Repair step
- Manager never saw "Proceed to Repair"
- JobAssignmentServiceImpl.startJob could flip REPAIR_PENDING → REPAIR_IN_PROGRESS itself
- RepairTask completion precondition was commented out

AFTER

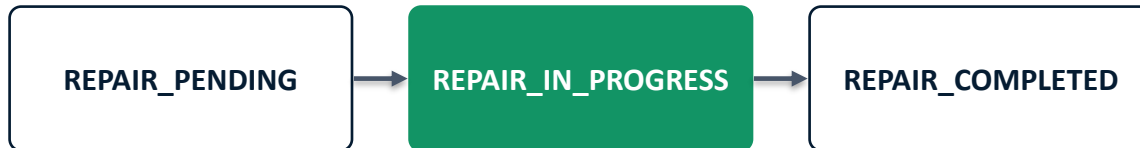
- APPROVAL completes only at REPAIR_IN_PROGRESS
- Manager stays on Approval: "Customer Approved — Manager Confirmation Required"
- ApprovalScreen refreshes with resumeWorkflow, re-derives the step (no local nextStep())
- startJob refuses unless the job is REPAIR_IN_PROGRESS
- completeRepair requires REPAIR_IN_PROGRESS again
- resolveStep aligned to shell numbering (Approval 8, Repair 9)

Key files: ServiceWorkflowServiceImpl.resolveCompleted · WorkflowController._resolveCurrentStepFromCompletedSteps · JobCardServiceImpl.startRepair · JobAssignmentServiceImpl.startJob



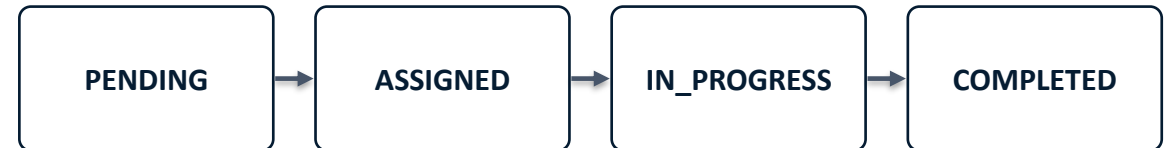
JobCard vs RepairTask

JobCard = overall service lifecycle



- Moved to REPAIR_IN_PROGRESS only by the manager gate
- Moves to REPAIR_COMPLETED automatically when EVERY task is COMPLETED (validated)
- QC failure sends it back to REPAIR_PENDING

RepairTask = one unit of repair work

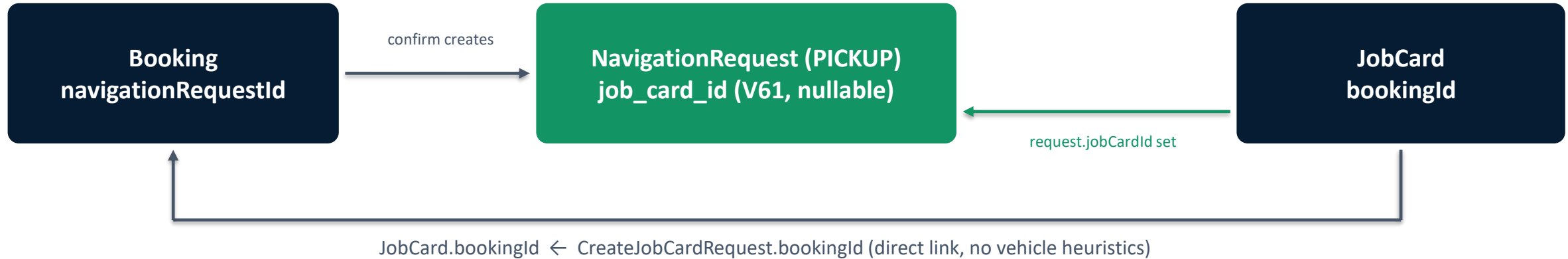


- Created on customer approval (createRepairTasks)
- Assigned through JobAssignment; technician accepts, starts, completes
- Start/complete require the JobCard to be REPAIR_IN_PROGRESS
- ON_HOLD / CANCELLED are declared-only

Task start DOES NOT change the JobCard. The only task → job coupling is the automatic all-tasks-complete aggregate.



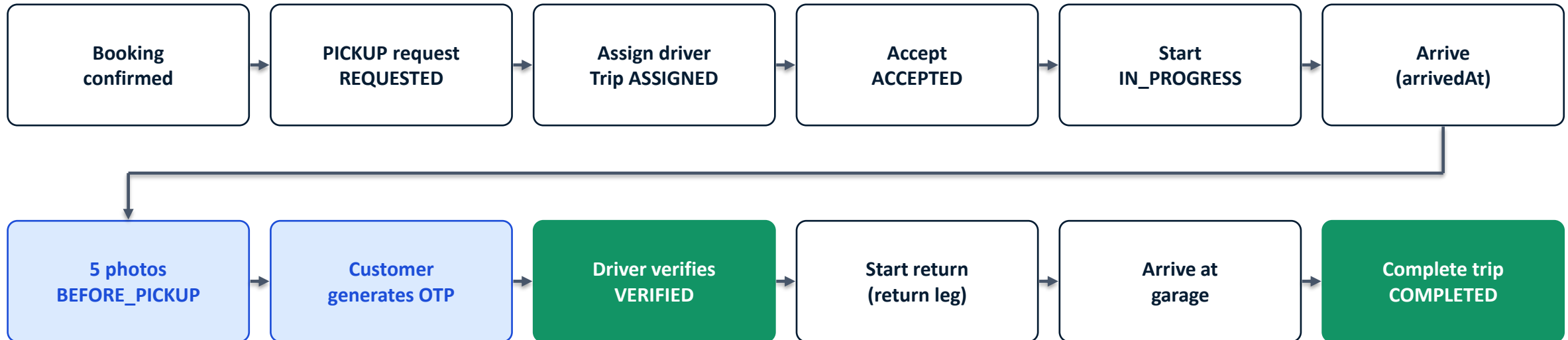
Booking → NavigationRequest → JobCard



- BookingDetailScreen "Create Job Card" preloads customer + vehicle, sets WorkflowState.bookingId and opens the workflow on the Job Card step
- JobCardServiceImpl.resolveBooking: same garage, CONFIRMED, same vehicle/customer, no other job uses it
- Then: JobCard.bookingId set, Booking → COMPLETED, pickup request gets job_card_id — all in one transaction
- Walk-in jobs (no bookingId) are unchanged



Pickup architecture

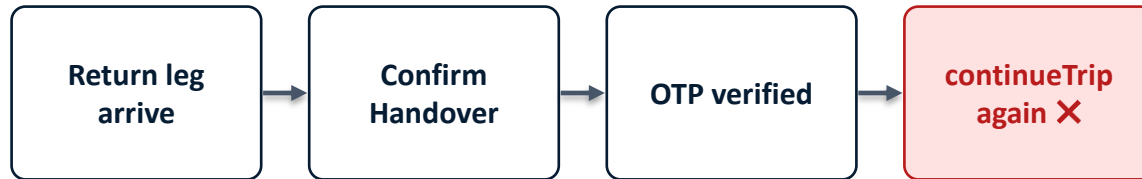


- Request status: REQUESTED → ASSIGNED → IN_PROGRESS (start) → COMPLETED (complete)
- Return leg: no evidence and no OTP again — the customer handover was verified before the return leg could start
- No garage-side OTP by design: there is no counter-party in the app at the garage
- Afterwards staff create the job card from the booking (job linked, booking COMPLETED)



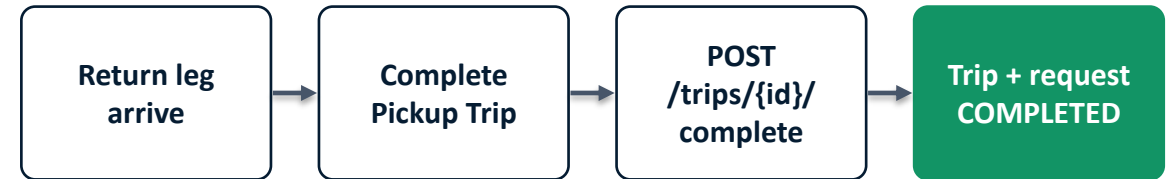
Pickup return-leg fix

OLD



- No UI ever called `completeTrip` for pickup
- Backend rejected the second continue ("already on the return leg")
- Pickup trips could not reach COMPLETED

NEW

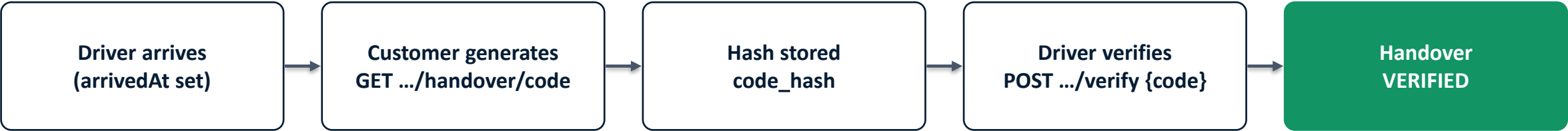


- `NavigationTrip.nextDriverAction` → `completePickup`
- `completeTrip` needs: return leg, arrivedAt, required photos, VERIFIED handover
- OTP code/verify are refused on the return leg

Backend detail: on the return leg `destinationAddress` may still be "GARAGE"; `ActiveTripScreen` resolves the real garage from `/garages/{id}`.



OTP architecture

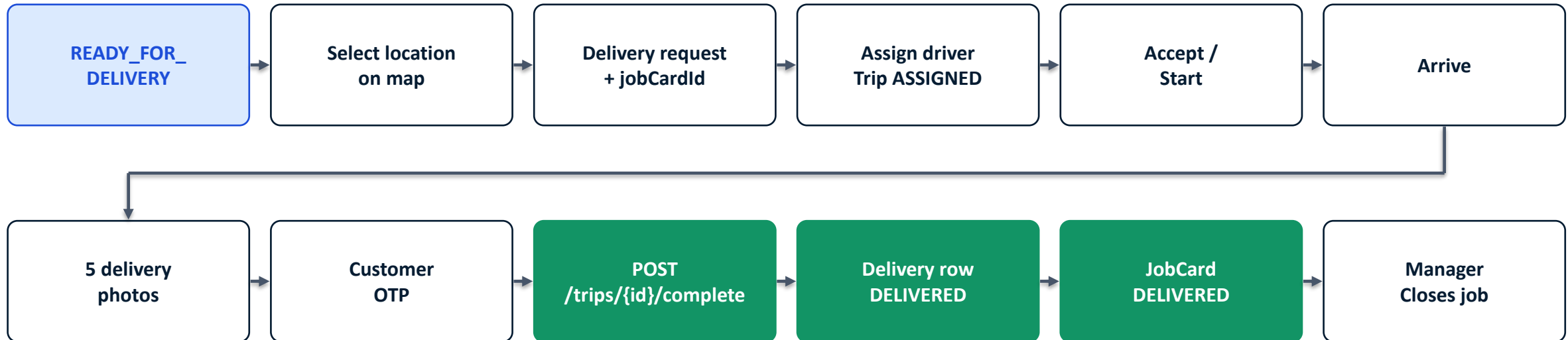


Aspect	Behaviour
Code	6 digits, SecureRandom; only a PasswordEncoder hash is stored
Expiry	10 minutes; a new request expires the previous pending code
Attempts	5 failed attempts → code EXPIRED, customer generates a new one
Direction	trip type: PICKUP or DELIVERY — identical flow
Delivered to driver	shown in the customer app and read out — not sent by SMS
Gates	continueTrip (pickup) and completeTrip (delivery) require VERIFIED

Handover states: PENDING → VERIFIED | EXPIRED (CANCELLED declared-only)



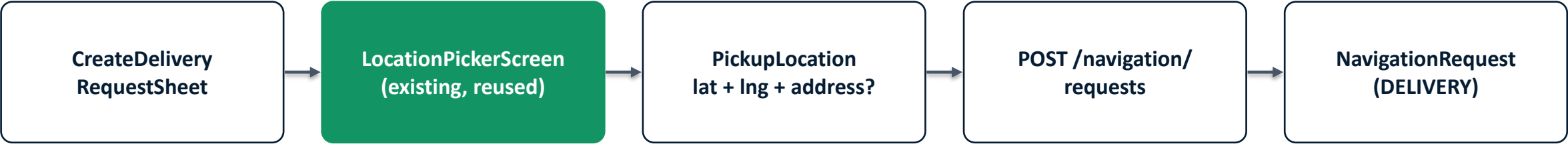
Delivery architecture



- The delivery TRIP is the business completion: OTP + evidence prove the handover
- Manual POST /deliveries remains only for a customer collecting at the garage — rejected once a delivery request exists
- Delivery screen restores its state from the backend (GET /navigation/requests/job-card/{id}/delivery)
- Not implemented: auto-assignment, driver rejection, reassignment, cancellation



Delivery location: map selection

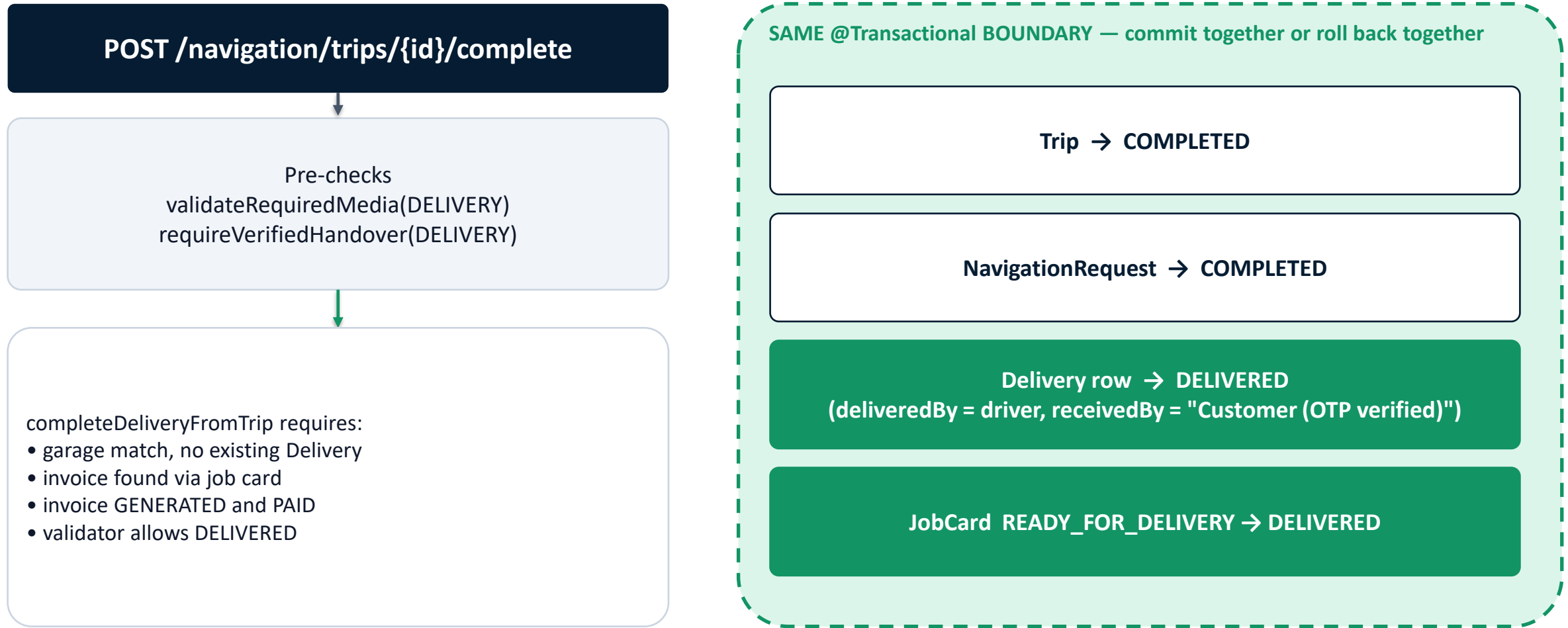


Field	Stored in	Rule
deliveryAddress	navigation_request.delivery_address	required; falls back to "lat, lng" if no street address resolves
deliveryLatitude / deliveryLongitude	navigation_request.delivery_latitude / _longitude	both required (server-side validation)
jobCardId	navigation_request.job_card_id	required for DELIVERY; job must be READY_FOR_DELIVERY

One picker for pickup and delivery. The trip destination for DELIVERY trips comes from these coordinates → NavigationTrip.destinationLatitude/Longitude → ActiveTripScreen route. The "use current location" shortcut remains.



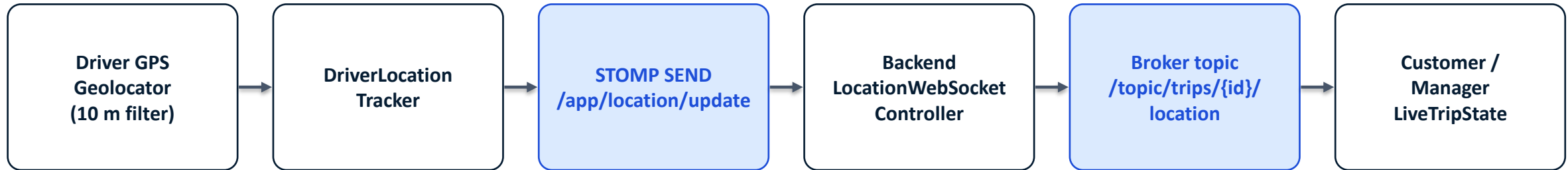
Delivery completion is one transaction



Legacy delivery requests (no job_card_id): the trip completes, no job effect.



Live navigation architecture

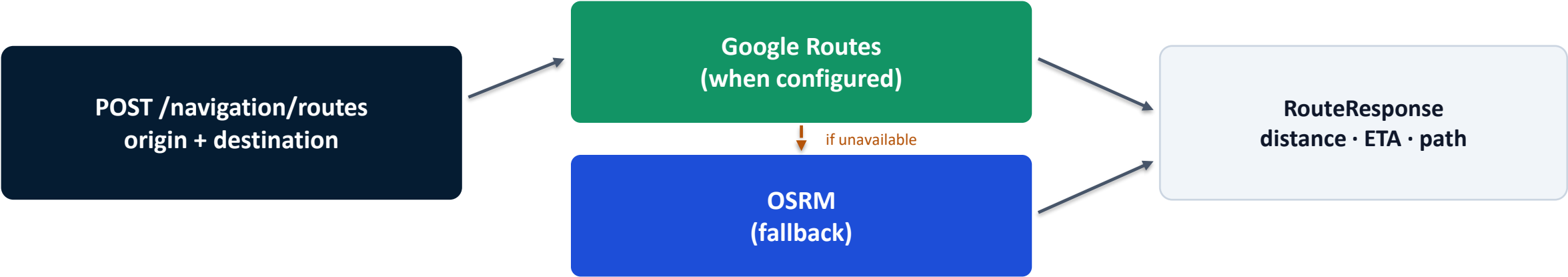


REST FALLBACK

- LiveTripState polls GET /navigation/trips/{id} every 10 s and reads GET .../location when no recent socket fix
- Socket: endpoint /ws, prefix /app, broker /topic; reconnect every 5 s; topics re-subscribed
- Tracking runs while ActiveTripScreen is open for the driver
- Subscription authorized per trip (TripLocationTopicInterceptor → NavigationTripAccessGuard)



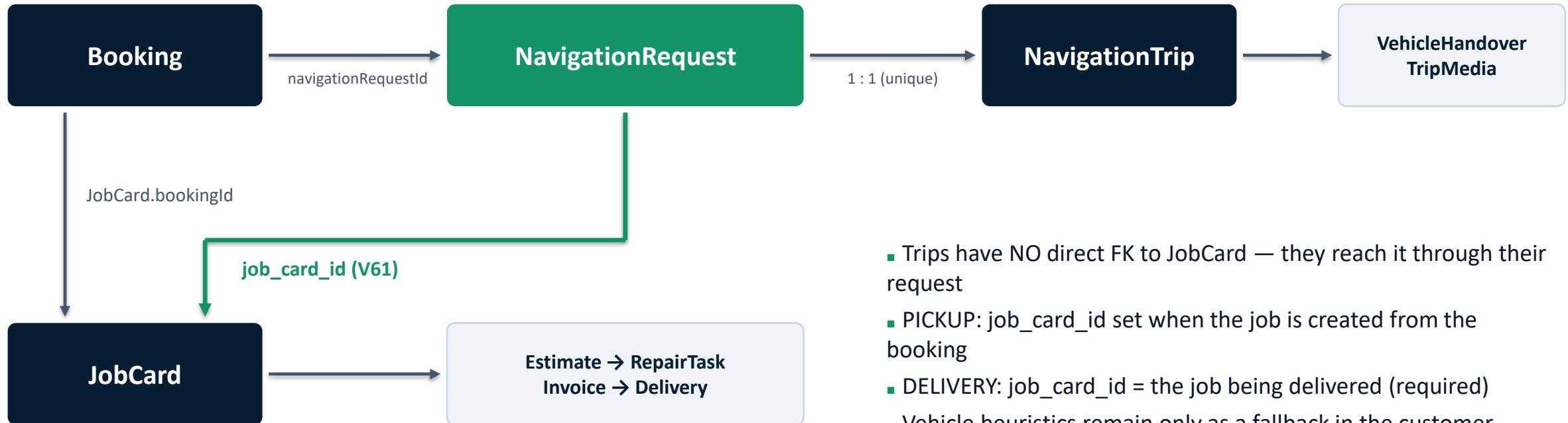
Routing and recalculation



Parameter	Value (from code)
Driver location filter	distanceFilter 10 m (DriverLocationTracker)
Route recalculation	only after ≥ 150 m movement (_routeRecalculationDistanceMeters)
REST poll fallback	10 s (_statusPollInterval)
Arrival	backend gate = driver POST /arrive; 10 m GPS test is display only
Destination	pickup: pickup coords · delivery: delivery coords · return leg: garage from /garages/{id}



Trip ↔ Booking ↔ JobCard relationships



- Trips have NO direct FK to JobCard — they reach it through their request
- PICKUP: `job_card_id` set when the job is created from the booking
- DELIVERY: `job_card_id` = the job being delivered (required)
- Vehicle heuristics remain only as a fallback in the customer journey



Customer journey selection

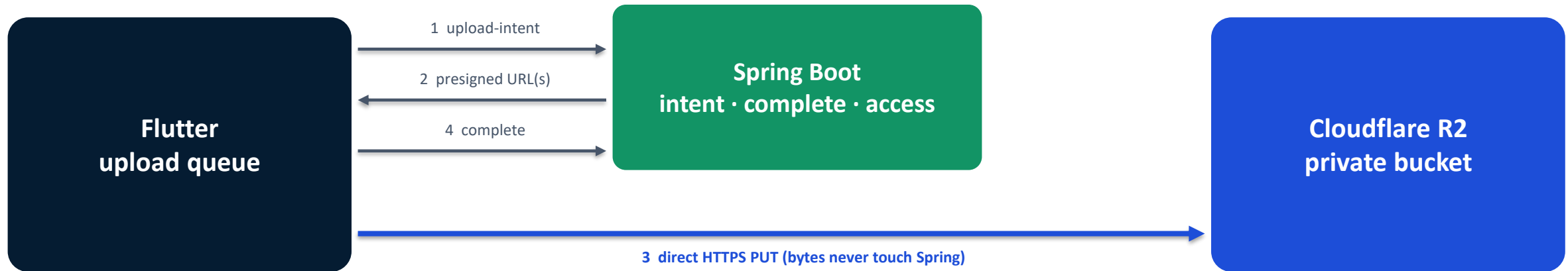
Priority	Source (per vehicle)	Stages
1	Active trip (ASSIGNED / ACCEPTED / IN_PROGRESS)	DRIVER_ASSIGNED → DRIVER_EN_ROUTE → DRIVER_ARRIVED → EN_ROUTE_TO_GARAGE; HANDOVER (delivery)
2	Non-terminal job card (not CLOSED/CANCELLED)	ARRIVED_AT_GARAGE ... VEHICLE_READY, COMPLETED (DELIVERED)
3	Pending request (REQUESTED)	BOOKING_CONFIRMED
4	Confirmed booking	BOOKING_CONFIRMED

Linked-pickup exception: if the active trip is a PICKUP whose request.job_card_id equals the vehicle's job card, the trip is ignored for stage selection — the vehicle is at the garage, the job card is the truth. A DELIVERY trip keeps priority.

- Edge cases: an unlinked, uncompleted pickup trip still shows EN_ROUTE_TO_GARAGE; vehicle-based selection can mispair overlapping records



Media architecture: direct to R2

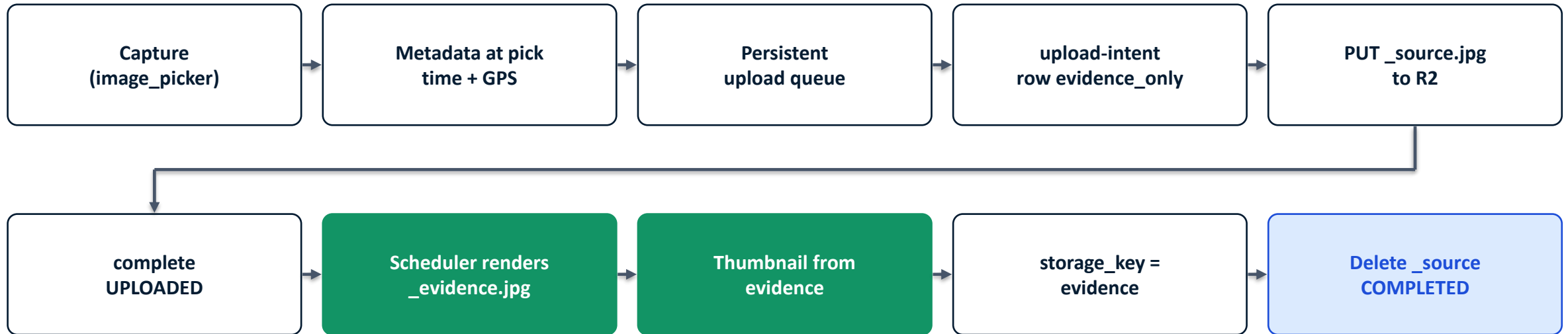


No Spring multipart proxy. The old multipart limit was not raised. Private bucket: every read/write is a short-lived presigned URL.

- NEW media = evidence + thumbnail. There is no separate original asset.
- Presigned TTL: upload 15 min (default), playback 10 min (default)
- Legacy Drive / original-based media keeps working (evidence falls back to the stored original)



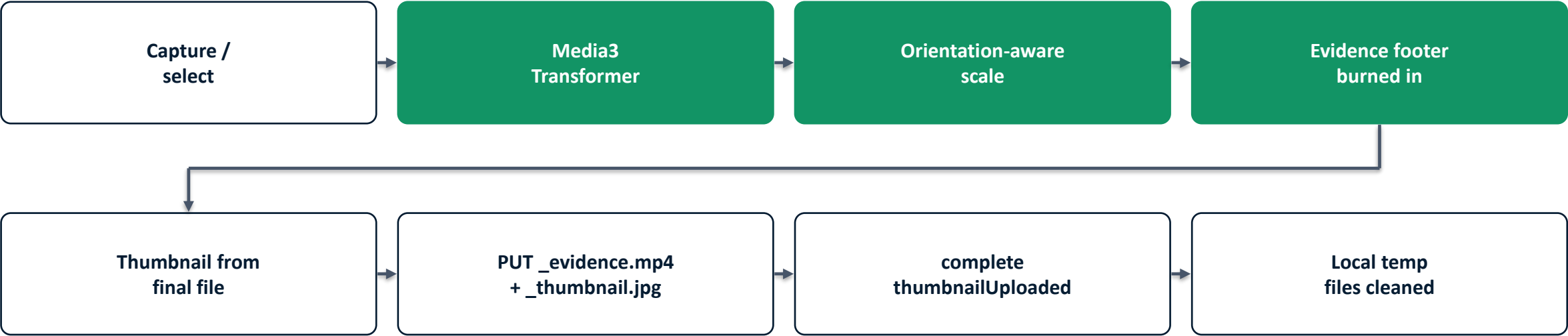
New image pipeline



- Footer: stage, captured time, captured-by, address, Lat/Lng, accuracy, GarageST — from persisted capture metadata (never current time/location)
- Retry: failure keeps the row UPLOADED, retry_count++, FAILED after 5 (source kept); a lost save after rendering is not re-rendered
- Save precedes source deletion: a crash can only orphan the temporary source
- R2 key: GarageST/{garageCode}/{jobCardNumber}/{stage}/{jobCardNumber}_{stage}_{seq}_{source|evidence|thumbnail}.{ext}



New video pipeline



Parameter	Value
Max dimensions	long edge ≤ 1920, short edge ≤ 1080 (in the video's own orientation; even sizes)
Bitrate	80 % of the source bitrate (qualityPercent 80), H.264 / AAC MP4
Passes	single re-encode (optimization + overlay together)
Thumbnail	JPEG, longest edge ≤ 480 px, extracted from the FINAL file

Retry never re-renders once paths are persisted; cache key media-video-{mediaId} is stable. Video quality/size: MANUAL device validation.



JobCard media vs trip media

	JobCardMedia	NavigationTripMedia
Owner	a JobCard	a NavigationTrip
Storage	Cloudflare R2 (Drive for legacy)	server local disk (Config-dependent durability on Render)
Upload	presigned direct PUT + persistent queue	multipart through Spring
Processing	evidence footer + thumbnail	none
Metadata	captured-by/at, lat/lng, accuracy, address	shot type, stage, lat/lng, capturedBy/At
Purpose	inspection / repair evidence	pickup / delivery condition evidence (5 required shots gate trip steps)

Keep them separate. If a link is ever needed, connect by ids — never share code paths or duplicate media.



Database: V61

navigation_request
... existing columns ...
job_card_id BIGINT NULL
(no default, no foreign key)

Index	Definition
idx_navigation_request_job_card	on (job_card_id)
idx_navigation_request_one_delivery_per_job	UNIQUE on (job_card_id) WHERE request_type='DELIVERY' AND status<>'CANCELLED' AND job_card_id IS NOT NULL

- Why the partial unique index: database backstop for at most one live delivery request per job (concurrent submits)
- Required by: new DELIVERY requests (service-level). PICKUP gets it later from the booking; legacy rows stay NULL and valid
- Rollback: additive and nullable — an older backend runs fine; a new backend on the old schema fails ddl-auto=validate. Undo needs a V62; never edit V61
- Flyway applies it at startup before JPA validation. NOT executed against PostgreSQL in this environment



Notification schema (additive)

device_token	notification	notification_outbox	notification_delivery
user_id → users token (UNIQUE) platform · app_version is_active · last_seen_at	user_id → users type · category · priority title · message · entity · job_card_id event_key · is_read · read_at UNIQUE (event_key, user_id)	event_key (UNIQUE) event_type · garage_id · entity payload (TEXT, ids only) status · attempt_count next_attempt_at · locked_at	notification_id → notification device_token_id → device_token status · attempt_count next_attempt_at · sent_at UNIQUE (notification, device)

Indexes	Purpose
uk_notification_event_user	idempotency: one logical notification per (event, recipient)
idx_notification_user_created	history paging (user_id, created_at DESC, id DESC)
idx_notification_user_unread (partial)	fast unread count: WHERE is_read = FALSE
idx_notification_outbox_status_next / ..._delivery_status_next	worker polling by (status, next_attempt_at)

Rollback: tables are additive; remove only with a NEW migration. Operational rollback = NOTIFICATIONS_ENABLED=false. Verified by NotificationPipelineIT on the isolated garageos_test database.



Screen → endpoint → service → state

Flutter	Endpoint	Service method	State written
CustomerEstimateDetailScreen	PUT /estimates/{id}/approve	EstimateServiceImpl.approveEstimateCanonical	Estimate APPROVED, tasks, job REPAIR_PENDING
ApprovalScreen	POST /workflow/{n}/repair/start	JobCardServiceImpl.startRepair	JobCard REPAIR_IN_PROGRESS
RepairScreen	PUT /job-assignments/{id}/start complete	JobAssignmentServiceImpl.startJob / completeJob	task/assignment; job REPAIR_COMPLETED (auto)
BookingDetailScreen / JobStepScreen	POST /workflow/job (bookingId)	JobCardServiceImpl.createJobCard	OPEN; Booking COMPLETED; request job_card_id
DriverDashboard	POST /navigation/trips/{id}/accept start arrive continue complete	NavigationTripServiceImpl	trip + request status
HandoverCode / VerifyScreen	GET .../handover/code; POST .../verify	HandoverServiceImpl	VehicleHandover PENDING → VERIFIED
CreateDeliveryRequestSheet	POST /navigation/requests	NavigationRequestServiceImpl.createRequest	NavigationRequest DELIVERY REQUESTED
HandoverVerifyScreen (delivery)	POST /navigation/trips/{id}/complete	DeliveryServiceImpl.completeDeliveryFromTrip	Delivery DELIVERED; job DELIVERED
Media queue	POST /job-cards/{id}/media/upload-intent; /media/{id}/complete	MediaServiceImpl	JobCardMedia rows



Transaction boundaries

Customer approval	EstimateServiceImpl.approveEstimate	estimate + RepairTasks + job REPAIR_PENDING
Manager confirmation	JobCardServiceImpl.startRepair	job REPAIR_IN_PROGRESS
Repair completion	RepairTaskServiceImpl / JobAssignmentServiceImpl.completeJob	task + assignment sync + job aggregate
Booking → job	JobCardServiceImpl.createJobCard	job + Booking COMPLETED + request job_card_id
Delivery trip completion	NavigationTripServiceImpl.completeTrip	trip + request + Delivery + job DELIVERED

Not one transaction: Flutter's job-create → inspection-start chain (two calls) and media processing (idempotent R2 writes, save before source delete).

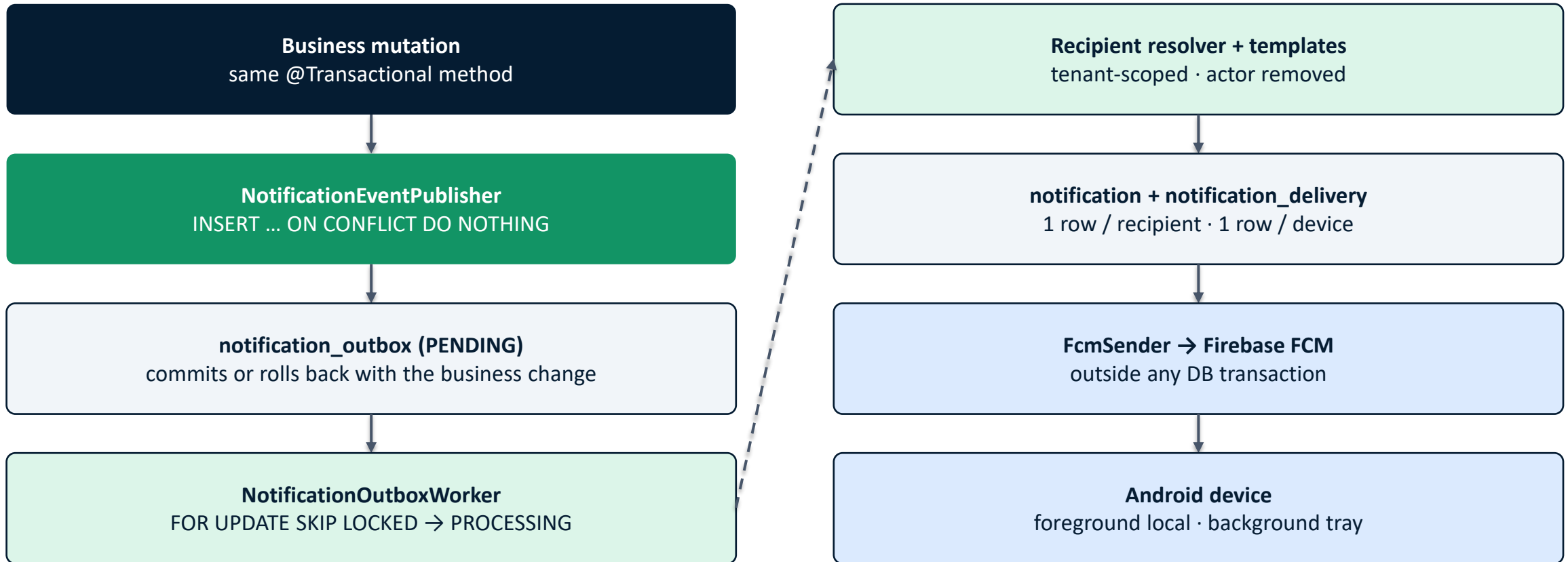


Legacy compatibility

Component	Status	Behaviour / rule
Google Drive media + multipart upload	Legacy	still served via /content; new code uses the R2 flow
R2 rows with original + thumbnail/evidence	Legacy	evidence falls back to the stored original; nothing migrated or deleted
JobCard states ESTIMATE_APPROVED, WORK_COMPLETED, INVOICED, PAYMENT_PENDING, PAYMENT_COMPLETED	Legacy	never written; old rows can still move forward
Delivery requests without job_card_id	Legacy	trip completes, no job effect
/workflow/... vs /jobcards/... duplicate endpoints	Duplicate	delegate to the same rule
JobAssignmentType.DRIVER	Legacy	no writer/caller found; use trips
Declared-only statuses (Estimate SENT/EXPIRED, RepairStatus ON_HOLD/CANCELLED, NavigationRequest/Trip CANCELLED, DeliveryStatus PENDING/READY, ...)	Declared-only	never written
flutter_compress dependency	Unused	no Dart imports; Media3 declared directly



Business event → notification pipeline

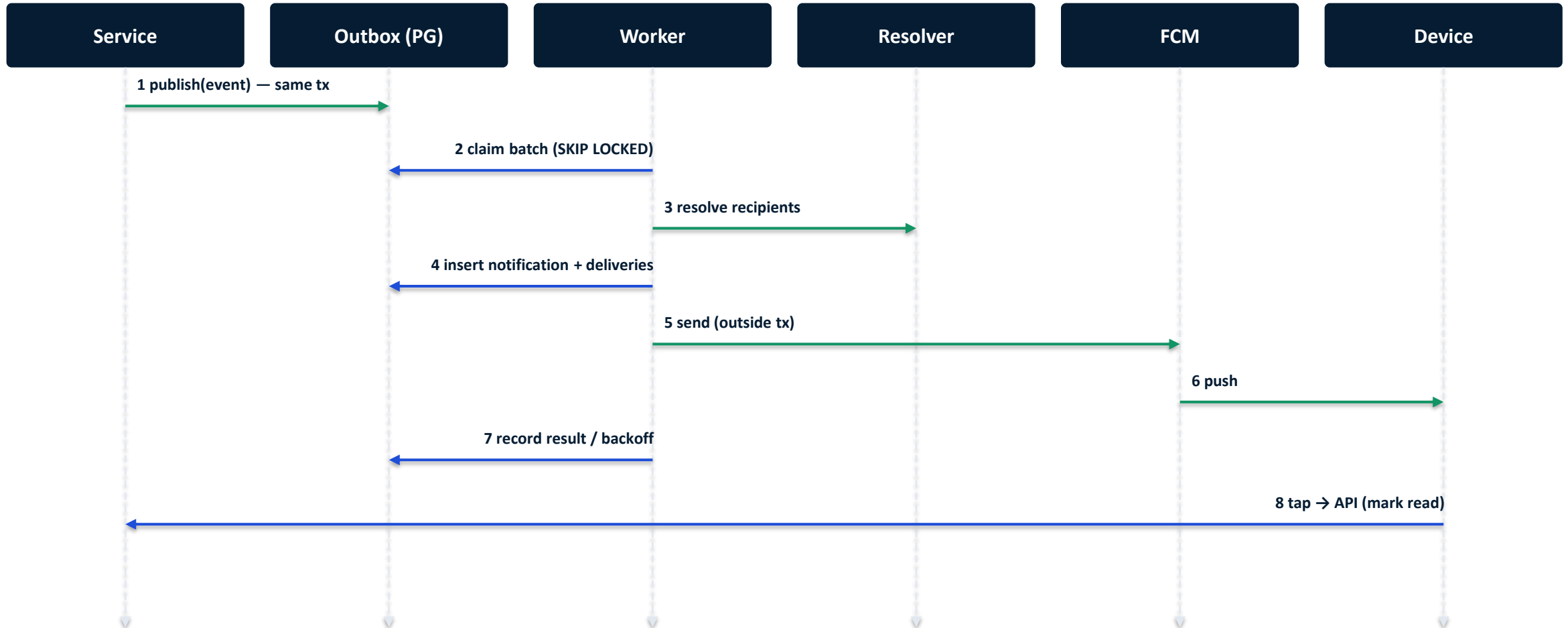


Tap → NotificationNavigationService (queued until session restored) → NotificationDestinationResolver → GarageST screen

At-least-once delivery · PostgreSQL = source of truth · FCM = delivery only · no Kafka · no DB triggers · WebSocket stays live-trip only.



End-to-end sequence



Rollback of step 1 removes the event (no phantom notification). An FCM outage never rolls back a business operation — steps 5–7 retry independently.



Notification events 1–16

Event	Source (where the transition is finalised)	Recipients	Category / priority	Entity
BOOKING_REQUESTED	BookingServiceImpl.createBooking	Owner, Manager/SA	BOOKING / NORMAL	BOOKING
BOOKING_CREATED_BY_STAFF	BookingServiceImpl.createPhoneBooking	Customer	BOOKING / LOW	BOOKING
BOOKING_CONFIRMED	BookingServiceImpl.confirmBooking	Customer	BOOKING / NORMAL	BOOKING
BOOKING_REJECTED	BookingServiceImpl.rejectBooking	Customer	BOOKING / NORMAL	BOOKING
BOOKING_CANCELLED	BookingServiceImpl.cancelBooking	Customer, Manager/SA	BOOKING / NORMAL	BOOKING
PICKUP_DRIVER_ASSIGNED	NavigationTripServiceImpl.assignDriver	Driver, Customer	TRIP / HIGH	TRIP
TRIP_ACCEPTED	NavigationTripServiceImpl.acceptTrip	Customer, Manager/SA	TRIP / NORMAL	TRIP
TRIP_STARTED	...startTrip / continueTrip	Customer, Manager/SA	TRIP / NORMAL	TRIP
DRIVER_ARRIVED	...arriveAtDestination	Customer	TRIP / HIGH	TRIP
HANDOVER_CODE_GENERATED	HandoverServiceImpl.getOrCreateActiveCode	Driver	TRIP / NORMAL	TRIP
HANDOVER_VERIFIED	HandoverServiceImpl.verify	Customer, Manager/SA	TRIP / NORMAL	TRIP
TRIP_COMPLETED	NavigationTripServiceImpl.completeTrip	Customer, Manager/SA	TRIP / NORMAL	TRIP
JOB_CARD_CREATED	JobCardServiceImpl.createJobCard	Customer, Owner	JOB_CARD / NORMAL	JOB_CARD
INSPECTION_STARTED	JobCardServiceImpl.startInspection	Customer	INSPECTION / LOW	JOB_CARD
INSPECTION_COMPLETED	JobCardServiceImpl.completeInspection	Customer, Manager/SA	INSPECTION / NORMAL	JOB_CARD
ESTIMATE_READY_FOR_APPROVAL	EstimateServiceImpl.updateEstimate	Customer	ESTIMATE / HIGH	ESTIMATE

Acting user is always excluded. Idempotency key = TYPE:ENTITY_TYPE:ENTITY_ID:discriminator (transition time for re-enterable states).



Notification events 17–32

Event	Source (where the transition is finalised)	Recipients	Category / priority	Entity
ESTIMATE_APPROVED	EstimateServiceImpl.approveEstimateCanonical	Customer, Manager/SA, Owner	ESTIMATE / HIGH	ESTIMATE
ESTIMATE_REJECTED	EstimateServiceImpl.rejectEstimate	Manager/SA, Owner	ESTIMATE / HIGH	ESTIMATE
REPAIR_TASK_ASSIGNED	JobAssignmentServiceImpl.assignJob/reassignJob	Technician	ASSIGNMENT / HIGH	REPAIR_TASK
REPAIR_TASK_STARTED	RepairTask...startRepair / JobAssignment...startJob	Manager/SA	REPAIR / LOW	REPAIR_TASK
REPAIR_TASK_COMPLETED	RepairTask...completeRepair / JobAssignment...completeJob	Manager/SA	REPAIR / LOW	REPAIR_TASK
JOB_REPAIR_STARTED	JobCardServiceImpl.startRepair	Customer	REPAIR / NORMAL	JOB_CARD
JOB_REPAIR_COMPLETED	all-tasks-complete transition (3 sites)	Customer, Manager/SA, Owner	REPAIR / NORMAL	JOB_CARD
QUALITY_CHECK_STARTED	ServiceWorkflowServiceImpl.performQualityCheck	Manager/SA	QUALITY_CHECK / LOW	JOB_CARD
QUALITY_CHECK_PASSED	QualityCheckServiceImpl.passQualityCheck	Manager/SA	QUALITY_CHECK / NORMAL	JOB_CARD
QUALITY_CHECK_FAILED	QualityCheckServiceImpl.failQualityCheck	Technician, Manager/SA, Owner	QUALITY_CHECK / HIGH	JOB_CARD
INVOICE_GENERATED	InvoiceServiceImpl.createInvoice/generateInvoice	Customer, Manager/SA, Owner	INVOICE / HIGH	INVOICE
INVOICE_ACCEPTED	InvoiceServiceImpl.acceptInvoice	Manager/SA, Owner	INVOICE / NORMAL	INVOICE
PAYMENT_RECEIVED	InvoiceServiceImpl.receivePayment	Customer, Manager/SA, Owner	PAYMENT / NORMAL	INVOICE
VEHICLE_READY_FOR_DELIVERY	JobCardServiceImpl.readyForDelivery	Customer, Manager/SA, Owner	DELIVERY / HIGH	JOB_CARD
VEHICLE_DELIVERED	DeliveryServiceImpl (both paths)	Customer, Manager/SA, Owner	DELIVERY / NORMAL	JOB_CARD
JOB_CLOSED	JobCardServiceImpl.closeJobCard	Owner	JOB_CARD / LOW	JOB_CARD

Acting user is always excluded. Idempotency key = TYPE:ENTITY_TYPE:ENTITY_ID:discriminator (transition time for re-enterable states).



Persona-based notification routing

Customer own bookings, trips, estimate, repair, invoice, payment, delivery Resolved from: customer → mobile number → user (role CUSTOMER). No app account = no push	Owner business events: booking, job card, estimate, repair, QC fail, invoice, payment, vehicle ready/delivered, closed Resolved from: garage.owner_user_id	Manager · Service Advisor operational updates: bookings, trip, inspection, estimate, repair tasks, QC, invoice, payment Resolved from: users.garage_id = event garage, role MANAGER / SERVICE_ADVISOR
Technician REPAIR_TASK_ASSIGNED · QUALITY_CHECK_FAILED Resolved from: assignee ids captured in the event (before QC reset)	Driver PICKUP_DRIVER_ASSIGNED · HANDOVER_CODE_GENERATED Resolved from: trip.driverId, same garage	No automatic events Accountant · Cashier · Inventory manager Resolved from: not part of the notification matrix

Actor self-notification is suppressed (a manager approving an estimate is not told about it). Recipients are de-duplicated; the first persona wins.



Multi-garage isolation



Staff recipients (owner, manager, service advisor, technician, driver) must belong to the event's garage — same tenant source as the security principal (users.garage_id).

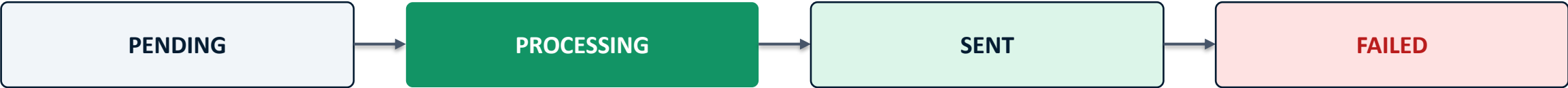
A technician or driver id from another garage supplied through event facts is rejected (tests: technician_isResolvedFromFacts_onlyWithinGarage, crossGarage_staffOfGarageB_neverReceiveGarageAEvents).

Customers are cross-garage records: they receive events only for their OWN customer id (they may legitimately hear from several garages).

The unscoped getTechnicians() / getRepairEmployees() queries are never reused. FCM topics are not used; recipients are chosen server-side.



Transactional outbox and retry



FAILED after 6 attempts · stale PROCESSING (> 5 min) → PENDING

Setting (application.properties)	Current value / behaviour
notifications.enabled	false by default (NOTIFICATIONS_ENABLED). Off = publisher no-op, worker never starts
worker.delay-ms · batch-size	5 000 ms · 50 rows per batch; FOR UPDATE SKIP LOCKED claim
worker.max-attempts · base-backoff-seconds	6 attempts · 30 s doubling (30 s, 60 s, 2 m, 4 m ...) capped at 1 h
worker.stale-lock-minutes	5 — crashed PROCESSING rows are recovered
delivery lease	2 minutes — a claimed delivery is invisible to other claims
FCM timeouts	connect 10 s · read 15 s; FCM is never called inside a DB transaction

Idempotency: outbox event_key UNIQUE · notification (event_key, user_id) UNIQUE · delivery (notification, device) UNIQUE. Evidence: NotificationPipelineIT (rollback leaves no row; replay creates no duplicate).

Business rollback → no outbox row. FCM down → business operation still succeeds; delivery retries independently.



FCM architecture and configuration



Priority	Android channel	Used for
HIGH	garagest_urgent (high importance, FCM priority HIGH)	estimate ready/approved/rejected, task assigned, QC failed, invoice, vehicle ready, driver arrived/assigned
NORMAL	garagest_default	most progress events
LOW	garagest_low	inspection started, repair task start/done, QC started, job closed, staff booking

Configuration — names only

NOTIFICATIONS_ENABLED (default false)

FIREBASE_CREDENTIALS_BASE64 (Render secret)

Payload (identifiers only)

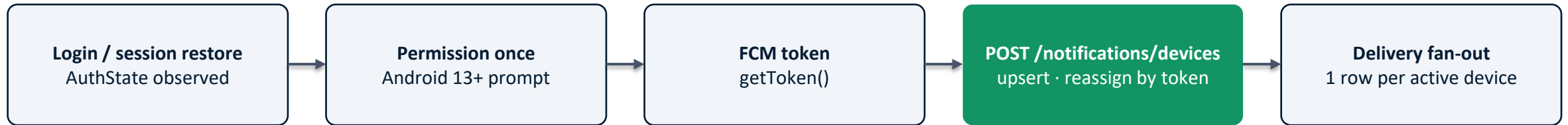
notificationId · type · entityType · entityId · jobCardId · garageId

no names, phones, amounts or tokens

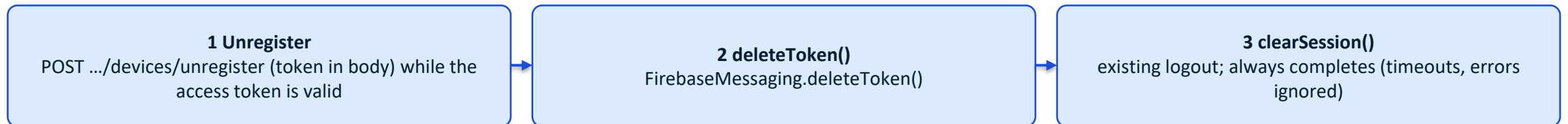
No credentials → NoopFcmSender: app starts, history still works, no pushes. Invalid token (UNREGISTERED) → deactivated. Auth/permission errors pause the tick, never touch business data.



Device token lifecycle



Logout (required order — AuthState.onBeforeLogout)



Token = app installation, attached to the authenticated user who last registered it. Re-registration by another user reassigns it (account switching).

FCM token refresh → register again (signed in only). Multi-device: one row per installation; per-device delivery status.

UNREGISTERED → delivery INVALID_TOKEN and token deactivated. No refresh interceptor and no last_seen_at pruning exist (limitation).



Foreground · background · terminated

Foreground	Background	Terminated / cold start
FlutterMessaging.onMessage	system tray shows the FCM notification block	getInitialMessage() or local launch payload captured
de-duplicate by notification id	Dart background handler = no-op	queued — NOT dispatched yet
FlutterLocalNotifier.show (channel by priority)	tap → onMessageOpenedApp	dispatch after: session restored · signed in · past splash/login
badge + list refresh (NotificationState)	NotificationNavigationService.handleTap	then role-aware navigation

Tap safety: mark-read doubles as the ownership check (404 = not this user's → nothing opens). Duplicate taps within 5 s ignored; queued taps expire after 30 min; destinations re-check access through the API.

Permission denied is respected: no tray notification, but history and badge still work.



Notification history and deep linking

Persona	Entity	Destination (existing route unless marked new)
Customer	BOOKING	Routes.bookingDetail
Customer	TRIP	DRIVER_ARRIVED → handoverCode · HANDOVER_VERIFIED → handoverStatus · else activeTrip
Customer	ESTIMATE / INVOICE	customerEstimateDetail / customerInvoiceDetail (new routes)
Customer	JOB_CARD / REPAIR_TASK	customerJobCard (new) → resolves job number → CustomerRepairTrackingScreen
Owner · Manager/SA	BOOKING · TRIP · other	garageBookingQueue · managerFleetMap · jobCardDetail (job-card id)
Technician	any	technicianHome (no repair-task screen exists)
Driver	TRIP	HANDOVER_CODE_GENERATED → handoverVerify · PICKUP_DRIVER_ASSIGNED → driverHome · else activeTrip
any	ids missing	Routes.notifications (history)

History screen (/notifications)

title · message · relative time · unread dot
 paged (20) · pull to refresh · mark read · mark all read
 bell + unread badge in the 5 home shells

The payload is routing, not authorization

every destination loads data through the authenticated API
 backend authorization is unchanged
 server-side checks still decide access



Android notification experience

Area	Implementation
Gradle	com.google.gms.google-services 4.4.4 applied only if android/app/google-services.json exists · core-library desugaring
Dependencies	firebase_core ^4.15.0 · firebase_messaging ^16.7.0 · flutter_local_notifications ^22.3.1
Manifest	POST_NOTIFICATIONS · FCM default channel garagest_default · default icon @mipmap/ic_launcher
Channels	garagest_urgent (high) · garagest_default · garagest_low — created at startup; geolocator foreground service unaffected
Permission	Android 13+ runtime prompt once per app run after login; denial respected
Sound / vibration	default notification sound; vibration follows device mode and per-channel settings (OS/user control)
Platform status	Android tested · iOS NOT IMPLEMENTED (behind PushGateway, ready to add)

PHYSICALLY VALIDATED — ANDROID (product owner, real device, real Firebase)

notification appears with sound · vibrate/silent follows device mode · appears in the system tray · history visible in GarageST · unread badge works · tap opens GarageST with working deep link · Vehicle Ready for Delivery and Payment Received observed with job-card context
Not claimed: every one of the 32 events individually tested; iOS.



Testing and validation status

Backend tests	226 run • 3 known failures (InvoiceServiceImplTest.receivePayment_*, pre-existing, unrelated)
Flutter tests	368 passing
flutter analyze	0 errors • 2 warnings (pre-existing) • 94 info lints
APK (debug) / backend jar	PASS (jar with -DskipTests • debug APK)
Release AAB	NOT RUN
V61 on PostgreSQL	ENVIRONMENT REQUIRED
Device flows	MANUAL — notification flows PHYSICALLY VALIDATED on Android

Not covered by automated tests: start_repair projection filter; DeliveryScreen restoration (widget test).



Manual test matrix (key scenarios)

ID	Scenario	Expected result
WF-004	Customer approves; manager opens app	Approval screen says Customer Approved — Manager Confirmation Required; repair NOT started
WF-005	Manager confirms	JobCard REPAIR_IN_PROGRESS; Repair step opens
PK-004	Pickup return	route to the garage; no OTP/evidence asked
PK-005	Pickup completion + job from booking	trip/request COMPLETED; booking COMPLETED; job linked
DL-001 ... DL-006	Delivery: request → map → assign → navigate → OTP → complete	Delivery DELIVERED; job DELIVERED → Close Job; reopening the step restores state
NAV-001 / 002	STOMP updates / REST fallback	marker moves; fallback ≈ every 10 s
MED-001 ... 004	Image, video, customer playback, legacy media	evidence + thumbnail only in R2; legacy plays

Full 25-row matrix (WF, PK, DL, NAV, MED) is in handbook section 38.



Notification testing and validation

Area	Evidence	What it proves
Publisher / idempotency	7 unit	stable event key · replay = same key · disabled = no-op · actor captured
Recipient resolver	13 unit	customer with/without account · all personas · actor suppression · cross-garage rejection
Templates · device tokens · history	4 + 7 + 5 unit	no PII · token reassignment/ownership · user-scoped history, mark-read
Outbox · retry · FCM results	15 + 4 unit	claim · stale recovery · backoff · invalid token deactivation · config-error pause
Real database	6 IT (PostgreSQL)	rollback leaves no outbox row · replay de-duplicated · full chain · user isolation
Flutter	50 tests	routing · cold-start queue · token lifecycle · logout ordering (real AuthState) · API contract

AUTOMATED VALIDATION
backend 226 run (223 pass; 3 pre-existing InvoiceServiceImplTest)
NotificationPipelineIT 6/6 · Flutter 368 pass
backend jar + debug APK built
uses a stub FCM sender — no Firebase contact

PHYSICAL DEVICE VALIDATION — ANDROID
real FCM · sound · tray · history · badge · tap + deep link
pending manual: cold start per persona, permission denied, multi-device,
account switch, all 32 events



Architectural rules — do not break these

1 JobCard ≠ RepairTask

2 Customer approval ≠ manager confirmation; only startRepair opens repair

3 Backend state is authoritative; no local step advance past a refresh

4 Every JobCard.status writer uses JobCardStatusValidator

5 A delivery completes through its delivery trip

6 Trip / delivery state never lives only in Flutter

7 Direct-to-R2 media; no Spring proxy; no larger multipart limit

8 New media = evidence + thumbnail; legacy stays compatible

9 No secrets in source control

10 Never modify an applied Flyway migration

11 Keep JobCardMedia and NavigationTripMedia separate

12 No garage-side OTP; no second continueTrip on the return leg



Invariants 13–17 — do not break these

13

Publish events inside the business transaction through `NotificationEventPublisher`, only from meaningful transitions. Never call FCM from a business transaction; no DB triggers; no “watch every UPDATE”.

14

Every staff recipient must belong to the event's garage. Recipient rules live only in `NotificationRecipientResolver`. Never use FCM topics as authorization.

15

FCM payload carries identifiers only; notification text carries no customer/vehicle/financial data; never log FCM tokens or credentials.

16

Logout must unregister and delete the device token BEFORE the session is cleared (`AuthState.onBeforeLogout`); cleanup must never block logout.

17

Changing a notification event requires updating the matrix (handbook §28A.4), the resolver, the template factory and the tests.



Architecture decisions (ADR-001 ... 008)

ADR	Decision	Consequence
001 Direct R2	presigned PUT to a private bucket + persistent client queue	client owns retry; R2 config required
002 Evidence canonical	only evidence + thumbnail; images server-rendered, videos on-device (Media3)	two renderers kept in sync by test literals
003 JobCard vs RepairTask	separate machines; task actions need REPAIR_IN_PROGRESS	technicians blocked until confirmation
004 Approval vs confirmation	REPAIR_PENDING = approved, awaiting manager; APPROVAL completes at REPAIR_IN_PROGRESS	no new status; resolveCompleted encodes it
005 Trip ↔ JobCard	one nullable navigation_request.job_card_id	legacy rows NULL; no FK
006 Delivery trip = delivery	trip completion creates Delivery and marks the job DELIVERED (one transaction)	requests need jobCardId
007 Map delivery location	reuse LocationPickerScreen; address + lat + lng required	old apps cannot create delivery requests
008 FCM + outbox notifications	PostgreSQL transactional outbox → worker → Firebase FCM; history in PostgreSQL; no Kafka	at-least-once delivery; client de-duplicates; Firebase secret to manage; iOS later



Current limitations and engineering risks

V61 not verified on PostgreSQL

ENVIRONMENT REQUIRED

Release AAB not built

NOT RUN

JWT default secret + test DB password in source control

rotate / move to environment

Trip media on server local disk

durability is config-dependent on Render

No cancellation, rejection or reassignment

a stuck trip never auto-expires

Customer journey vehicle-based fallback

can mispair overlapping records

Flutter never auto-refreshes an expired access token

known gap

Return-leg backend destination stays "GARAGE"

Flutter resolves the real garage



Notification limitations

iOS push NOT IMPLEMENTED

no APNs / Firebase iOS setup

Customer ↔ user match by mobile number

no FK; no account = no push

Legacy backend logout never revokes the session

device-token cleanup is a separate explicit call

No refresh interceptor / no last_seen pruning

expired-session tokens cleaned on re-register or UNREGISTERED

Launcher icon reused as notification icon

may render white on some Android versions

Automated tests use a stub FCM sender

real delivery covered by the physical test

createInvoice has no @Transactional (pre-existing)

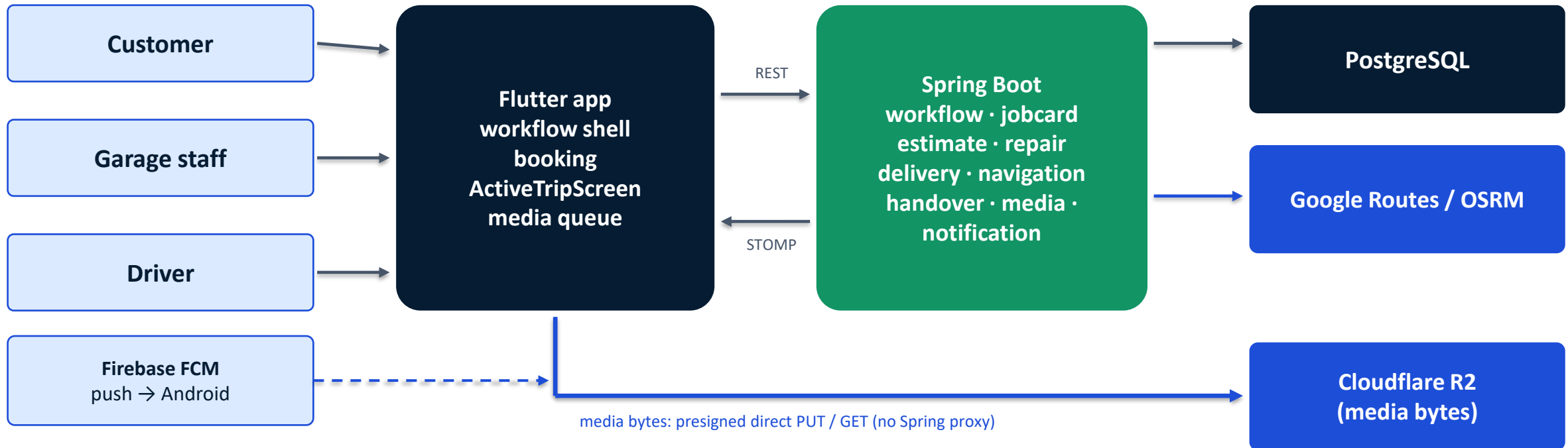
its outbox row is not atomic on that path

No retention job · no preferences · appVersion not sent

tables grow; every relevant event pushed



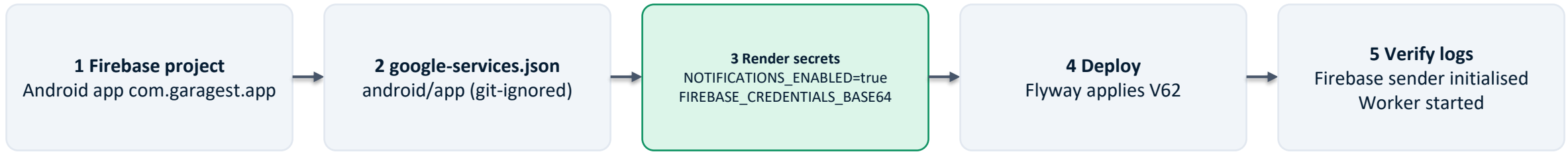
Final system map



Lifecycle: Booking → Pickup trip → Job Card → Inspection → Estimate → Customer approval → MANAGER confirmation → Repair → QC → Invoice → Payment → Delivery request → Delivery trip (OTP + evidence) → Delivery / job DELIVERED → Closed. Push notifications (FCM) inform the right user at each stage.



Enabling and rolling back notifications



Variable names only — never commit or document values (service-account JSON, keys, tokens).

Disabled by default: `NOTIFICATIONS_ENABLED=false` → publisher no-op, worker never starts, no FCM calls.

Enabled without credentials: app still starts; in-app history only; no pushes.

Rollback: set `NOTIFICATIONS_ENABLED=false` and restart. V62 tables stay; any removal needs a new migration.

Optional tuning keys: `notifications.worker.delay-ms` · `batch-size` · `max-attempts` · `stale-lock-minutes` · `base-backoff-seconds`



Notification security

Identity	JWT principal only; no user id accepted from the client	History	WHERE user_id = caller; mark-read looks up (id, user_id) → 404 otherwise
Devices	unregister only own token; register reassigns by token	Tenant	staff recipients must share the event's garage
Customer	events only for the customer's own entities	Payload	identifiers only; text carries only the job-card number
Secrets	service account only via FIREBASE_CREDENTIALS_BASE64; never in Flutter or source	Logs	no raw FCM tokens; ids only ([NOTIF_OUTBOX] [NOTIF_RESOLVE] [NOTIF_FCM] [NOTIF_DEVICE])



2026-10-02 — Push Notification System

Backend module notification: 32 events, recipient resolver (garage isolation, actor suppression), persona templates

V62: device_token · notification · notification_outbox · notification_delivery (idempotency constraints)

Transactional outbox + retry worker (own executor, SKIP LOCKED, backoff, stale-lock recovery) · Firebase Admin SDK 9.9.0

APIs: device register/unregister · history · unread count · mark read · mark all read · rejectEstimate now @Transactional

Flutter: firebase_messaging + local notifications · cold-start queue · role-aware deep links · history screen · unread badge · ordered logout cleanup

Android: Google Services plugin (conditional) · channels urgent/default/low · runtime permission

Validation: backend 226 (223 pass) · IT 6/6 · Flutter 368 · PHYSICALLY VALIDATED on Android (partial scope)

Limitations: iOS not implemented · mobile-number customer match · legacy logout bug · stub FCM in automated tests



Maintaining the handbook

1 Read the relevant section before changing architecture

2 Inspect the actual code first; if they disagree, the code wins and the handbook is fixed

3 Update the handbook in the same task as any architectural change

4 Add or amend an ADR for every significant decision

5 Update state-machine tables when a transition, writer or guard changes

6 Update the API reference when a business endpoint, role or validation changes

7 Add recurring problems to the debugging runbook and error table

8 Never document secrets — variable names only

9 Never document unimplemented behaviour as current

10 Keep legacy behaviour labelled with removal criteria

Regenerate the PDF and this deck after every handbook change.